The Null Is Part of the Instrument: Failure Modes in AI-Assisted Experimental Software Research

Physis Research Program

September 24, 2026

Abstract

AI agents that design, implement, and interpret their own experiments introduce systematic errors that human-only research does not face. We catalog twelve documented failure modes from a three-month AI-assisted research program, organized by the phase of research they affect: design (injection policy, fixture circularity), implementation (harness bugs, parser confounds), and interpretation (confound checks that stop early, baseline drift). In every case, a construction-matched null — a control specifically designed to perturb the claimed mechanism — either exposed the error or would have. We propose that null design is the primary instrument in AI-assisted empirical research, not a formality, and that its quality determines what can be concluded.

1 Introduction

AI-assisted research agents can run hundreds of experiments, iterate on code, and interpret results at a speed that amplifies both productivity and error. When an agent designs an experiment, implements it, runs it, and then interprets the output — all without human intervention — the classical experimental safeguards (peer review, pre-registration, replication) are weakened or absent.

We report from a three-month program in which AI agents autonomously designed, implemented, and interpreted experiments across five research tracks: ontology induction, coding-agent behaviour, semantic trajectories, contradiction detection, and retrieval. The program produced a settled negative (cross-embedder ontology agreement is near chance) and several positives (cross-agent failure transfer, semantic trajectories), but it also produced a dozen experiments that looked positive and were wrong.

We catalog these failures not as embarrassment but as data. Each failure mode is a property of the research process when an agent is both experimenter and interpreter. Our central claim: **the construction-matched null is the primary instrument**, and its design determines what can be concluded.

2 Failure Modes

2.1 Design Failures

D1. Wrong injection distribution (E14). An experiment measured scorer AUC at 0.880 for detecting misfiled ontology entries. The injection policy placed errors at uniformly random cells. When the injection was changed to the entry's nearest alternative cell — a plausible error — AUC dropped to 0.506, at chance. The scorer detected distance moved, not wrongness.

D2. Generated fixture circularity (E83). A contradiction-detection rule was evaluated on fixtures generated by inserting WordNet antonyms into a template. The rule achieved AUC 0.998. On hand-written fixtures without lexical antonyms, the same rule achieved recall 0.143. The generator was grading itself.

D3. Adversarial collapse (E85). A structured rule set achieved precision 1.000 and recall 0.840 on a benign holdout fixture. On adversarial fixtures designed to fire each rule wrongly, precision collapsed to 0.000. All four false positives had the same shape: class cues matched without binding them to the pair's participants.

D4. Scale-dependent controls (E19b). A control arm that was valid at 4B model scale became invalid at 9B: the control converged on the same answer as the treatment, not because the effect was real but because the model was too weak to distinguish the conditions.

2.2 Implementation Failures

D5. Harness parsing bugs (E74). Cline's shell results arrive as batched output under `result/output/error` keys. Reading only the `text` key returned nothing for shell actions, producing a failure rate of 0.005 — which reads as a very reliable agent rather than as a parser that found no output. The bug was detected only by looking at the rates.

D6. Duplicate sessions inflating N (E73). Transcripts sync between machines. A byte-identical session present on three machines would be counted three times, inflating the corpus to 1.6x its true size. The bug was detected by checking session IDs, not by trusting the file count.

D7. Degenerate null (E78, first run). A trajectory experiment embedded only the first 1200 bytes of each revision — the module header, which barely changes. Twelve revisions produced identical vectors, the within-object distance was 0.0000, and the direction null collapsed to +/-1.000. The null was measuring a constant function.

D8. Measurement parser bug (E76). A coherence-tracking script matched "the first number after the word coherence," which picked up the node count and reported 4.0 -¿ 5.0 as a coherence rise. The parser was doing the measuring.

2.3 Interpretation Failures

D9. Confound check that stops at the first reassuring answer (E17). ANCHORVOC's confound was tested three ways. Test 1 passed (the scorer predicted the rater's cell choice). Test 2 passed (it survived half the anchor's hints). Test 3 failed (it rose on hint-leaking entries where cosine fell). The script's first verdict read only Test 1, found it passed, and declared the confound resolved — ignoring that Test 3 had failed a criterion the same script had pre-registered.

D10. Baseline drift across runs (E87). A pooled-vector baseline scored 0.400 in one run and 0.200 in the next. The tree moved under the experiment: repos grow, and deterministic sampling of a growing directory produces different samples. The report cited the 0.400 figure; the rerun in the same script produced 0.200. Only the in-run comparison is valid.

D11. Metric field mismatch (E81). A pipeline that reported "no structure" was averaging a working rule set (ratio 3.57) with a broken one (ratio 0.92). The aggregate 0.96 said no structure; the decomposition said multi-word cues work (15:1 over shuffle) and single-word cues do not (0.92:1).

D12. Pre-flight overridden by its author (E10). A power analysis returned STOP on an underpowered design. The run proceeded anyway. A control the author overrides is not a control.

3 Patterns

Three structural patterns emerge:

1. **Controls must be checked for validity, not just run.** D4 and D9 show that a control can be valid for one model and invalid for another, or pass one test and fail another. A control whose validity is not itself tested is a decoration.

2. **Baselines must be rerun in-run, not cited from reports.** D10 shows that a baseline can drift by 0.2 between runs. Any number cited from a previous experiment is an assertion about the past, not a measurement.

3. **Positive results are artifacts until a construction-matched says otherwise.** D1, D2, D3, and D8 all produced strong positives that disappeared under stronger controls. The controls that killed them were designed to perturb the claimed mechanism, not to confirm it.

4 Proposed workflow

From these failures we derive a minimal workflow for AI-assisted empirical research:

1. **Pre-register the null.** Before running, specify what result would falsify the claim. A null that cannot fail is not a null.

2. **Rerun baselines in the same execution.** Never cite a baseline number from a previous run. Compute it fresh, in the same script, on the same data.

3. **Test controls for validity.** A control arm that does not perturb the statistic it claims to control for is worse than no control: it produces a false sense of security.

4. **Decompose aggregates.** When a pipeline reports a single number, decompose it by component. D11 shows that a 0.96 average can hide a 15:1 success and a 0.92:1 failure.

5. **Inspect rates, not just counts.** D5 and D6 were both detected by looking at per-action rates (failure rate near zero, duplicate session IDs), not by trusting aggregate statistics.

6. **Retain failed branches.** A failed experiment is research evidence. The experiment ledger should preserve the full chain: hypothesis -¿ result -¿ control -¿ verdict -¿ revision.

5 Discussion

These twelve failure modes are not unique to this project. Any system that uses AI agents to design, implement, and interpret experiments is subject to the same classes of error. The contribution is the catalog, not the errors themselves.

The workflow we propose is deliberately minimal. It does not require pre-registration on a public registry, double-blind protocols, or statistical review boards. It requires only that the null be designed to kill the claim, that baselines be recomputed, and that failed branches be retained.

Limitations. The catalog is from one project, one user, one machine fleet. The frequency of each failure mode in other contexts is unknown. The proposed workflow has not been tested against a comparison group that does not use it. Both are open empirical questions.

6 Conclusion

The construction-matched null is not a formality in AI-assisted research. It is the primary instrument. Its design determines what can be concluded. Twelve documented failures — each caught by a null designed to kill the claimed mechanism, or by inspecting rates, or by rerunning baselines — demonstrate that the workflow is not optional.